

Analyzing Software Evolution through Design Pattern Evolution: A Preliminary Study

Stefania Celenta, Andrea De Lucia*, Vincenzo Deufemia, Carmine Gravino, Michele Risi

Dipartimento di Matematica e Informatica

Università di Salerno

84084 Fisciano(SA), Italy

stefaniacelenta@tiscali.it, {adelucia, deufemia, gravino, mrisi}@unisa.it

Abstract

In this paper we study the evolution of software systems at design level by analyzing the evolution of structural design patterns. In particular, the information on how the recovered patterns evolve in different versions of software systems is used to understand design and evolution decisions. Instances of design patterns are recovered by employing a two-phase recovery approach which is characterized by high precision values. We report on the application of the recovery process on five successive versions of JHotDraw library, which allow us to perform an initial preliminary study of JHotDraw evolution.

1. Introduction

The analysis of the evolution of software systems has been investigated in several contributions (see e.g., [2][3][10][12][15][21]). Some of the proposed approaches extract architectural information on software systems (see e.g., [2][11]). Others exploit information collected from the source code, such as those employing product metrics (see e.g., [21]).

One of the limitations of the existing software evolution techniques is that even if they correctly correlate software properties with the evolution of artifacts within a project, the obtained results are not extendable to other projects. In order to make more generalizable claims the software evolution analysis mechanisms have to deal with more abstract elements, which are sufficiently general and able to capture important aspects of the behavior of the software, so that they can be observed in multiple projects.

A design pattern represents a partial solution to a common non-trivial design problem, which highlights wished

properties of the design model. As a consequence, the extraction of design pattern information from source code helps the comprehension of the adopted solution for a software system. Thus, the analysis of the evolution of design pattern instances could be useful to understand the design decisions made by software designers to construct new versions of the software.

1.1 Related work

In the last years several methods have been proposed to address the problem of retrieving and identifying design patterns from source code and design documents. Most of the automatic or semi-automatic approaches presented in the literature are based on the availability of structural information about the patterns or equivalently a pattern library represented in some way [1][4][5][16][20][22][23].

To our knowledge no quantitative study has been published about the use of design patterns to analyze software evolution. Only few works can be mentioned, which differs from our approach since they consider the notion of piece of design patterns. In [14][19] the authors analyzed micro patterns that can be considered at an abstract level between design patterns and implementation patterns. In particular, they investigated the evolution of the changes in micro patterns to highlight areas of software projects that are more bug-prone. To this aim categories of micro pattern evolution were identified and their frequency in the software is associated to the bug rates. This analysis is used to identify those areas that requires more testing and verification resources. Following the same base line, in [7] Cinnéide and Nixon proposed an automatic approach to obtain design patterns from mini patterns by applying mini transformations in successive versions of a software system. A mini pattern is a motif that occurs frequently across the

*Corresponding author:

Via Ponte Don Melillo, 84084 Fisciano (SA)

Tel: +39089963376, fax: +39089963303

catalogues of design patterns defined in [13].

1.2 Our proposal

In this paper we investigate the evolution of software systems by analyzing the evolution of structural design patterns, which are detected by employing a two phase recovery approach. In the first phase, the class diagram is extracted from source code and design pattern instances are identified based on the design structure only by using a recovery technique inspired to visual language parsing [8]. In the second phase the identified candidate patterns are validated by performing a source code analysis, which eliminates many false positives and consequently increases the precision of the approach.

We report on the application of the recovery process on five successive versions of JHotDraw library [17]. By analyzing the recovered pattern instances of each version and identifying the kinds of structural changes characterizing them we are able to detect some of the design decisions made by designers.

The paper is structured as follows. The employed design pattern recovery process is briefly presented in Section 2. Section 3 reports the results of the design pattern recovery process carried out on five successive versions of JHotDraw library and the analysis of the design pattern evolution. Final remarks conclude the paper.

2. The employed design pattern recovery technique

In this section we describe the two-phase structural design pattern recovery approach that we employed to analyze the evolution of design patterns in software systems. The approach is depicted in Figure 1.

The Preliminary Analysis module extracts the structural information proper to recover design patterns and stores them into a database, so that they can be efficiently accessible and navigable. To accomplish this task we have used the code analysis tool Source Navigator [24], which is able to recover almost all the necessary information and organizes them in a data structure suitable for our purpose. The obtained

information can be easily analyzed to construct the corresponding UML class diagram and then visualized by a graphical editor.

The Structural Analysis identifies the candidate design patterns at a course-grained level by analyzing the class diagram structure obtained during the preliminary analysis. The recovery technique applied in the first phase is that integrated in the visual environment *DPRE* proposed in [8], which is based on visual language parsing. The recovery technique reduces the problem of identifying design patterns to the problem of recognizing subsentences in a class diagram, where each subsentence corresponds to a design pattern specified by a visual language grammar.

The goal of the Low-level Analysis module is to verify that the candidates are real instances of structural design patterns. To this aim, it analyzes the stereotypes associated to the classes, such as the *interface* stereotype, for validating the role of the participating classes and verifies the method declarations in the delegating classes. In particular, low-level checks are encoded into a sequence of queries carried out on the data structure obtained from the input source code during the preliminary analysis. Furthermore, the same kinds of checks are used to verify negative criteria requiring source code analysis.

A detailed description of the proposed two-phase design pattern recovery process can be found in [6].

3. Case study

In this section we describe a preliminary study of evolution of the design patterns, performed exploiting JHotDraw library. JHotDraw is a Java framework for drawing two-dimensional graphics in structured drawing editors [17], and can be used for developing custom-made drawing editor applications. JHotDraw was originally developed to illustrate the good use of design patterns for designing and documenting systems [18].

The aim of the study is to identify and analyze the kinds of changes characterizing structural design pattern instances across the analyzed JHotDraw versions, and relate them to decisions made on the software system by designers.

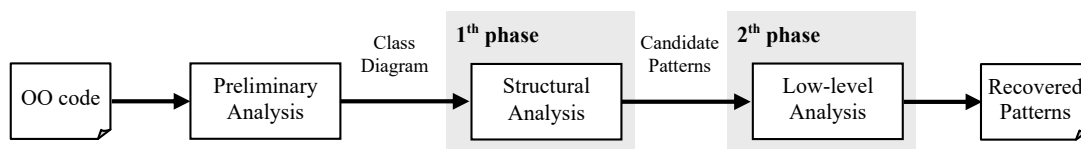


Figure 1. The design pattern recovery process

3.1 Design pattern recovery results

In this section we report the results of the structural design pattern recovery process applied on the versions from 5.1 to 6.0b1 of JHotDraw library [18]. Observe that 6.0b1 release is identical to the 5.4b2 release except for the package structure. Thus, we have only considered version 5.4b2.

Table 1 contains some statistics obtained from the software versions under analysis, while Table 2 summarizes the results of the recovery process.

Table 1. Some statistics obtained from five versions of JHotDraw

	5.1	5.2	5.3	5.4b1	5.4b2
#LOC	8419	9419	14601	28103	28398
#Classes	155	171	241	525	544
#DI	4612	5311	7963	12571	13079
#CRDPI	98	107	140	191	197

#DI = number of delegations and inheritances

#CRDPI = number of classes involved in the recovered design pattern instances

Table 2. Design pattern instances recovered in the analyzed JHotDraw versions

	5.1	5.2	5.3	5.4b1	5.4b2
#Adapter	41	42	37	54	53
#Bridge	75	81	98	151	166
#Composite	0	0	4	5	5
#Façade	9	9	14	19	20
#Proxy	0	0	0	0	0
#Decorator	0	0	0	0	0

For all the considered JHotDraw versions the proposed recovery approach has achieved a precision ranging from 0.82 to 0.93. Precision measure (together with the recall measure) is widely used to evaluate the retrieval effectiveness of a recovery strategy [1][5][22][23].

3.2 Design pattern evolution

From Table 2 we can observe that the number of design pattern instances is increased from version 5.1 to 5.4b2. However, the ratio between the number of classes and the number of structural design pattern instances is decreased from version 5.1 to version 5.4b2. In particular, in JHotDraw versions 5.1, 5.2, and 5.3 the number of classes involved in design pattern

instances is the 63%, 62%, and 58% of the total number of classes, respectively, while for versions 5.4b1 and 5.4b2 is 36%. Thus, we can deduce that from version 5.4b1 the incidence of structural design patterns is considerably decreased. On the contrary, for the first three analyzed JHotDraw versions a high percentage of changes concerned the insertion or modification or elimination of structural design pattern instances.

Table 3 reports some statistics on the evolution of design pattern instances across the analyzed JHotDraw versions. These statistics highlight that the changes in the number of design pattern instances are mainly due to the introduction of new classes. Moreover, the number of recovered design pattern instances present in all the analyzed versions is 53, while 25 are the instances not recovered after modifications in versions 5.3 and 5.4b1. In particular, 19 adapter pattern instances are not recovered in version 5.3 because a new abstract class has been interleaved between Target and Adapter classes, whereas in version 5.4b1 6 instances are not recovered because a method declared in the Target classes is eliminated. These pattern instances are valid because method's declaration is also present in the implemented interface.

We also observed that the more significant changes in the analyzed software systems were those involving instances of Adapter and Bridge patterns. However, due to the lack of the space, in this paper we report the results obtained by analyzing the evolution of Adapter pattern instances only. In particular, we analyzed the kinds of changes characterizing them across all the considered JHotDraw versions (see Table 4) and then we related them to the evolution activities performed by software designers.

From Table 4 we can note that new instances have been introduced by mainly adding new classes, while few pattern instances have been introduced by adding delegation relationships. Moreover, the Adapter and Adaptee classes are those more introduced in the software to obtain new Adapter pattern instances, whereas a Target class is always added together with other classes participating in an Adapter pattern. Just three instances of this kind of pattern have been removed during evolution of JHotDraw, in one case by eliminating a class and in two cases by modifying class relationships.

3.3 Evolution analysis

In the following we discuss the relation between the changes in the design pattern instances and the design decisions made by designers based on the kind of modification involving the patterns.

Table 3. Evolution of structural design pattern instances in successive versions of JHotDraw

	from 5.1 to 5.2	from 5.2 to 5.3	from 5.3 to 5.4b1	from 5.4b1 to 5.4b2
Introduced Design Patterns				
By introducing new classes	10	72	82	14
By modifying relationships	3	7	14	12
Eliminated Design Patterns				
By eliminating classes	-	-	3	1
By modifying relationships	2	33	5	5
Not Recovered Design Patterns	-	19	6	-
Recovered Design Patterns present in all versions	53			

Table 4. Evolution of Adapter Patterns

	from 5.1 to 5.2	from 5.2 to 5.3	from 5.3 to 5.4b1	from 5.4b1 to 5.4b2
Introduced Instances				
by introducing classes	2	12	23	-
# introduced classes	2	13	19	-
role of introduced classes	Adapter	8	14	-
	Adaptee	7	13	-
	Target	5	3	-
by modifying delegations	-	2	1	1
# modified delegations	-	9	2	1
Eliminated Instances				
by eliminating classes	-	-	1	-
# eliminated classes	-	-	1	-
by modifying relationships	1	-	-	2
# modified relationship	1	-	-	2
Not Recovered Instances	-	19	6	-

Insertion of an Adapter pattern instance. Table 5 shows the relation between the classes/delegations inserted for creating an Adapter pattern instance and the corresponding design evolution decisions made on JHotDraw. In particular, when a new pattern instance has been obtained by introducing in the software system a class playing the role of Adaptee we observed that a new feature were added to the system. This is due to the fact that the new functionalities provided by the legacy classes (i.e., the Adaptee) are exploited by the software system using the interface of the Target class. As an example, in version 5.3 of JHotDraw the pattern instance shown in Figure 2 has been introduced by inserting the classes VersionRequester (as Target) and VersionManagement (as Adaptee). In this case the new feature introduced by the pattern is a collection of utility methods to deal with version information (implemented by VersionManagement class).

When an Adapter pattern instance has been obtained by introducing in the software system a class playing the role of Adapter (also together with a class playing the role of Target) we observed that an existing feature was used by designers to provide a functionality with a customized interface. This corresponds to restructure the software in order to

make it more flexible and amenable to new requirements, which is the idea behind the definition of the Adapter pattern. In other words, the interface of a legacy class is converted into a different interface expected by the client, so that the client and the legacy class can work together without changes [13]. As an example, in version 5.3 the pattern instance shown in Figure 3 has been introduced by inserting the class FastBufferedUpdateStrategy with the role of Adapter.

Table 5. Design decisions characterizing the evolution of Adapter pattern instances in successive JHotDraw versions.

	Insertion of a new pattern instance	Elimination of a pattern instance
Adaptee	feature added	feature removed
Adapter	restructuring	-
Target + Adaptee	feature added	-
Target + Adapter	restructuring	-
Target + Adapter + Adaptee	feature added	-
delegation	restructuring	restructuring

In this case a double-buffering technique has been added to allow fast painting of the drawing area.

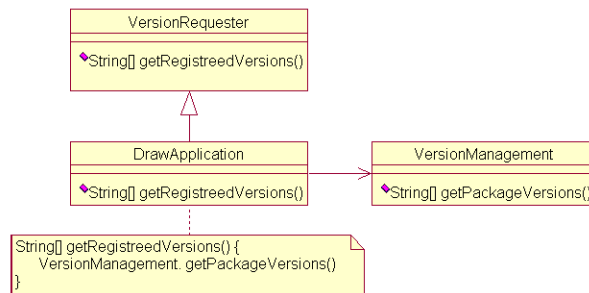


Figure 2. An instance of Adapter pattern by introducing the classes VersionRequester and VersionManagement

In few cases an Adapter pattern instance has been introduced by modifying delegations, only in versions 5.3 and 5.4b1. In these cases an existing class has been changed by introducing or modifying a delegation, so that it can play the role of Adapter for exploiting existing features.

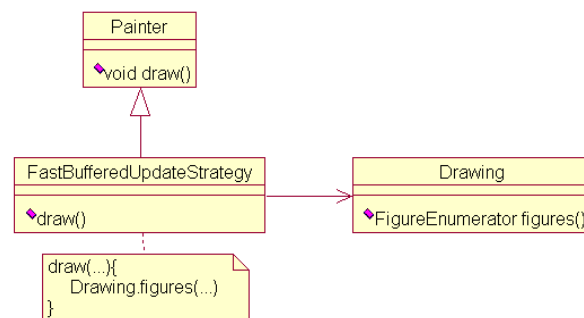


Figure 3. An instance of Adapter pattern obtained by introducing the class FastBufferedUpdatedStrategy

Elimination of an Adapter pattern instance. Table 5 also reports the relation between the classes removed (or the relationships modified) for eliminating an Adapter pattern instance and the design evolution decisions made on JHotDraw. In particular when a pattern instance has been eliminated by removing in the software system a class playing the role of Adaptee we observed that a feature was removed. As an example, in version 5.4b1 the feature that enumerated a vector back (size-1) to front (0) has been removed by eliminating the class **ReverseVectorEnumerator** from the system (see Figure 4). However, the analysis of the source code revealed that the

ReverseVectorEnumerator class has been substituted by the new class **ReverseListEnumerator** which provides an implementation of Java **Iterator** class.

In other few cases Adapter pattern instances have been removed by modifying relationships. For example, in two cases the hierarchical relationship between the class **ViewChangeListener** (playing the role of Target) and the classes **AbstractTool** and **AbstractCommand** (playing the role of Adapter) have been eliminated in version 5.4b2. The analysis of the source code revealed that the **ViewChangeListener** is directly instantiated inside the **AbstractTool** and **AbstractCommand** classes. Thus, it is clear that the designers have reconsidered the use of the Adapter pattern.

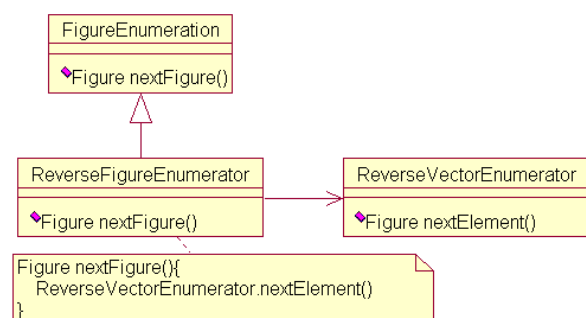


Figure 4. An instance of Adapter pattern removed from version 5.4b1 by eliminating ReverseVectorEnumerator

5. Final remarks

In this paper we investigate the evolution of software systems by analyzing the evolution of structural design patterns. These patterns are recovered by exploiting a two phase design pattern recovery process [6].

We have performed a preliminary study of JHotDraw evolution by analyzing the kind of changes characterizing structural design patterns. Indeed, the analysis revealed that the kind of changes in design pattern instances between two consecutive versions of JHotDraw can be used to detect some of the design decisions made by software designers. In particular, by observing how recovered Adapter pattern instances change we have highlighted some design decisions (feature added, feature removed, restructuring activity) made by software designers to evolve the system.

Our future investigations will be conducted in several directions. First, a more detailed and deeper analysis of the results obtained in this preliminary study should be performed also exploiting more documentation. Indeed, at moment we can exploit just

the source code comments. Moreover, we plan to verify whether there exist relationships between design pattern changes and the reliability of software (as an example, by analyzing bugs). Relationships between design pattern changes and complexity of software could also be analyzed. Finally, we plan to investigate more software systems to obtain other information on the impact of design patterns on software evolution.

References

- [1] G. Antoniol, G. Casazza, M. Di Penta, R. Fiutem, "Object-oriented Design Pattern Recovery", *The Journal of Systems and Software*, 59, 2001, pp.181-196.
- [2] G. Antoniol, G. Canfora, G. Casazza, A. De Lucia, "Maintaining Traceability Links during Object-oriented Software Evolution", in *Software, Practice, and Experience*, vol. 31, 2001, pp. 331-355.
- [3] G. Antoniol, M. Di Penta, E. Merlo, "An Automatic Approach to Identify Class Evolution Discontinuities", in *Proceedings of the 7th International Workshop on Principles of Software Evolution*, 2004, pp. 31-40.
- [4] F. Arcelli, S. Masiero, C. Raibulet, F. Tisato, "A Comparison of Reverse Engineering Tools based on Design Pattern Decomposition", in *Proceedings of the Australian Software Engineering Conference (ASWEC'05)*, Brisbane Australia, 2005, pp. 262-269.
- [5] Z. Balanyi and R. Ferenc, "Mining Design Patterns from C++ Source Code", in *Proceedings of International Conference on Software Maintenance (ICSM'03)*, Amsterdam, 2003, pp. 305-314, IEEE CS Press.
- [6] S. Celenta, A. De Lucia, V. Deufemia, C. Gravino, M. Risi, "A Two Phase Approach to Design Pattern Recovery". Technical Report (available from the authors)
- [7] M. Ó Cinnéide, P. Nixon, "Automated Software Evolution Towards Design Patterns", in *Proceedings of the International Workshop on the Principles of Software Evolution*, Vienna, September, 2001, pp. 162-165.
- [8] G. Costagliola, A. De Lucia, V. Deufemia, C. Gravino, M. Risi, "Case Studies of Visual Language Based Design Pattern Recovery", in *Proceedings of 10th European Conference on Software Maintenance and Reengineering (CSMR'06)*, IEEE CS Press, Bari, March 2006, pp. 163-172.
- [9] G. Costagliola, V. Deufemia, and G. Polese, "A Framework for Modeling and Implementing Visual Notations with Applications to Software Engineering", *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 13(4), 2004, pp. 431-487.
- [10] S. Demeyer, S. Ducasse, O. Nierstrasz, "Finding Refactorings via Change Metrics", in *Proceedings of Conference on Object-Oriented Programming, Systems, Languages and Applications*, 2000, pp. 166-177.
- [11] H. Gall, M. Jazayeri, R. Klosch, G. Trausmuth, "Software evolution observations based on product release history", in *Proceedings IEEE International Conference on Software Maintenance*, Bari, Italy, October 1997, pp. 160-166, IEEE CS Press.
- [12] H. Gall, M. Jazayeri, J. Krajewski, "CVS Release History Data for Detecting Logical Couplings", in *Proceedings of International Workshop on Principles of Software Evolution*, 2003, pp. 13-23.
- [13] E. Gamma, R. Helm, R. Johnson, J. Vlissides, "*Design Patterns: Elements of Reusable Object-Oriented Software*", Addison-Wesley, Menlo Park, CA, 1995.
- [14] J. Gil, I. Maman, "Micro Pattern in Java Code", in *Proceedings of Conference on Object-Oriented Programming, Systems Languages and Applications*, San Diego, USA, 2005, pp. 97-116.
- [15] M. Godfrey, L. Zou, "Using origin analysis to detect merging and splitting of source code entities", *IEEE Transactions on Software Engineering*, 31(2), 2005, pp. 166-181.
- [16] Y. Guéhéneuc, S. Hamel, O. Kaczor1, "Efficient Identification of Design Patterns with Bit-vector Algorithm", in *Proceedings of 10th IEEE European Conference on Software Maintenance and Reengineering (CSMR'06)*, Bari, 22-24, 2006, pp. 163-172, IEEE CS Press.
- [17] JHotDraw: <http://www.jhotdraw.org>
- [18] R. Johnson, "Documenting Frameworks using Patterns", in *Proceedings of Conference on Object-Oriented Programming, Systems Languages and Applications*, (OOPSLA'92), 1992, pp. 63-72.
- [19] S. Kim, K. Pan, E.J. Whitehead, "Micro Pattern Evolution", in *Proceedings of International Workshop on Mining Repositories*, May 22-23, Shangai, China, 2006.
- [20] C. Kramer, L. Prechelt, "Design Recovery by Automated Search for Structural Design Patterns in Object Oriented Software", in *Proceedings of Working Conference on Reverse Engineering (WCRE'96)*, 1996, pp. 208-215, IEEE CS Press.
- [21] M.M. Lehman, D.E. Perry, J.F. Ramil, "Implications of evolution metrics on software maintenance", in *Proceedings International Conference on Software Maintenance (ICSM'98)*, Bethesda, MD, 1998, pp. 208-217, IEEE CS Press.
- [22] J. Niere, W. Shafer, J. P. Wadsack, L. Wendehals, J. Walsh, "Towards Pattern design recovery", in *Proceedings of International Conference on Software Engineering (ICSE'02)*, Orlando Florida, USA, 2002, pp. 338-348.
- [23] I. Philippow, D. Streitferdt, M. Riebish, S. Naumann, "An Approach for Reverse Engineering of Design Patterns", *Software System Model*, 4(1), 2005, pp. 55-79.
- [24] SourceNavigator: <http://sourcnav.sourceforge.net/>